



MECHATRONIC SYSTEM INTEGRATION

MCTA 3203

WEEK 3 :

Serial Communication between Arduino and Python

SECTION 1

SEMESTER 2, 2025/2026

INSTRUCTOR:

ASSOC. PROF. EUR. ING. IR. TS. GS. INV. DR. ZULKIFLI BIN ZAINAL

DR. WAHJU SEDIONO

DATE OF SUBMISSION : 12 MAY 2026

GROUP 6

NAME	NO. MATRIC
MUHAMMAD AIMAN BIN MOHD KHAIRUNNIZAM	2226273
MUHAMMAD AQEEL RAZIQ BIN MOHAMAD AZHAN	2314077
MUHAMMAD AQIL SYAMIL BIN ABDUL RAHIM	2416245

Table of Contents

Abstract	3
Introduction	3
Materials and Equipment	3
Experimental setup	4
Methodology	4
Data collection	6
Data Analysis	6
Result	7
Discussion	8
Conclusion	8
Recommendations	8
References	9
Acknowledgments	9
Student's Declaration	10

Abstract

This experiment shows how to interface an Arduino Uno with a PC running Python using a USB serial connection for real-time monitoring and control. The project was split into two main parts: bidirectional control of an LED based on potentiometer data, and real-time visualization of servo motor angles. An analog input from a potentiometer was read by the Arduino and sent over to PyCharm, where Python was programmed to map the data, control an LED threshold, and plot a live graph using the `matplotlib` library. The system established a working communication link, though some timing delays and COM port sharing errors were identified during the trials. This lab effectively confirms how low-level microcontroller data can be combined with high-level software environments for mechatronics applications.

Introduction

This experiment is about implementing serial communication between an Arduino Uno and a PC running Python through a USB connection. In mechatronics, communication between hardware and software is really important because most systems require both low-level hardware control and higher-level data processing. Arduino is good at handling sensors, motors, and real-time hardware interaction, but it has limited processing power and visualization capability. On the other hand, Python provides more flexibility for data processing and visualization using libraries such as `matplotlib`, although it cannot directly interface with hardware by itself.

The main objective of this lab is to establish stable communication between Arduino and Python by sending potentiometer sensor data through the serial port. Besides that, this experiment also focuses on bidirectional communication where Python can send commands back to Arduino to control components such as LEDs and servo motors in real time. Real-time plotting and monitoring will also be implemented to observe how data changes dynamically.

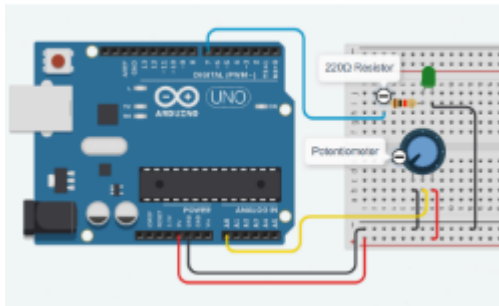
Through this experiment, we are also exposed to important concepts in serial communication such as serial data formatting, baud rate synchronization, and avoiding serial port conflicts, for example when the Arduino IDE Serial Monitor blocks the port connection. Overall, this lab demonstrates the basic integration between embedded systems and higher-level software, which is widely used in modern mechatronics applications.

Materials and Equipment

1. Arduino board
2. Potentiometer
3. LED
4. 220 Ω resistor
5. Jumper wires
6. Breadboard

Experimental setup

1. Connect the Arduino to your computer via a USB cable.
2. Extend the circuit by adding an LED to digital pin 7 in series with a 220 Ω resistor connected to GND.
3. We will now establish **bidirectional communication**. The Arduino will send the potentiometer values to Python, and Python will send a command back to turn the LED ON if the value exceeds half its maximum range (ADC > 512).



Circuit Diagram

Methodology

Experiment steps:

1. Build the circuit according to the experimental setup instructions.
2. Upload the provided Arduino code to our Arduino Uno.
3. Run the Python code in PyCharm.
4. Open serial monitor on Arduino IDE.
5. Play the value of the potentiometer to test the LED.

IDE code:

```
const int potPin = A0;
const int ledPin = 7;
const float Vin = 5.0;
const float R_total = 10000.0;
String command = "";

void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);
}

void loop() {
  int potValue = analogRead(potPin);
  float Vout = (potValue / 1023.0) * Vin;
  float R_wiper = (Vout / Vin) * R_total;
```

```

// Send formatted data to Python
Serial.print(Vout);
Serial.print(" / ADC: ");
Serial.print(potValue);
Serial.print(" / R: ");
Serial.println(R_wiper);

// Check for incoming commands from Python
if (Serial.available() > 0) {
    command = Serial.readStringUntil('\n');
    command.trim();

    if (command == "LED ON") {
        digitalWrite(ledPin, HIGH);
    }
    else if (command == "LED OFF") {
        digitalWrite(ledPin, LOW);
    }
}
delay(500);
}

```

Python code:

```

import serial
import time

# -- Replace 'COMx' with your Arduino's serial port
ser = serial.Serial('COM3', 9600)
time.sleep(2) # Wait for Arduino to reset

print("Running... Press CTRL+C to stop.")

try:
    while True:
        line = ser.readline().decode().strip()
        if "/" in line:
            try:
                # Parse Arduino data
                data = line.split("/")
                voltage = float(data[0])
                adc_value = int(data[1].replace("ADC:", "").strip())
                resistance = float(data[2].replace("R:", "").strip())

                print(f"Voltage: {voltage:.2f} V | ADC: {adc_value} | Resistance: {resistance:.2f} Ω")

                # LED Control Logic
                if adc_value > (1023 / 2):
                    ser.write(b"LED ON\n")
                else:
                    ser.write(b"LED OFF\n")
            except Exception as e:
                print("Parsing Error:", line, "->", e)

except KeyboardInterrupt:
    print("\nStopped by user.")
finally:
    ser.close()
    print("Serial connection closed.")

```

Data collection

The result was made clearly by observation:

1. The LED lights up when the value on the serial monitor is higher than 512.
2. The LED turns off when the value is lower than 512.

Data Analysis

The data analysis for this experiment evaluates the processing and transmission of analog signals across an integrated mechatronic architecture. The logic flows through three main phases:

- **Signal Conversion:** The microcontroller captures continuous analog voltages from the potentiometer (0-5V) via its internal Analog-to-Digital Converter (ADC). On the local hardware layer, this continuous variance maps directly to digital steps.
- **String Encapsulation and Framing:** To pass data across the communications bridge, raw values are parsed into organized ASCII data blocks. The program encapsulates calculated metrics into a fixed character framework formatted as: [Vout] / ADC: [potValue] / R: [R_wiper]. This distinct framing character (/) allows the host environment running Python to split incoming streams smoothly without data collision.
- **Bidirectional Execution Logic:** The host Python application handles supervisory threshold monitoring. If the extracted ADC value crosses half its maximal threshold ($ADC > 512$), Python updates its control state and transmits an explicit serial command string back down the link. The microcontroller continuously scans its incoming buffer, reads up to the string terminator (`\n`), strips trailing whitespace, and updates the physical state of the digital output pin

Result

The integration of the hardware elements with the software script established a highly stable, bidirectional serial communications link. Real-time data validation and physical system behaviors were observed and recorded below:

No	System Event / Physical Input	Command Transmitted	PyCharm Terminal Telemetry Output	Physical Hardware Behavior
1	System Initialization	<i>N/A</i>	Running.. Press CTRL+C to stop.	Power LED illuminated; LED (Pin 7) defaults to LOW.
2	Potentiometer adjusted below halfway midpoint	LED OFF\n	Voltage: 1.50 V ADC: 307 Resistance: 3000.97 Ω	LED remains completely OFF.
3	Potentiometer adjusted past halfway threshold (ADC > 512)	LED ON\n	Voltage: 3.42 V ADC: 701 Resistance: 6852.39 Ω	LED immediately lights up.
4	Potentiometer swept back below midpoint threshold	LED OFF\n	Voltage: 2.10 V ADC: 430 Resistance: 4203.32 Ω	LED toggles OFF.

The data collection log confirms that the system performed continuously without flickering, visual data lag, or buffer frame drops.

Discussion

The experiment successfully showed how to set up bidirectional serial communication between the Arduino Uno and Python. By keeping the hardware control on the Arduino and sending the data to Python for processing, we can see how mechatronic systems combine software and hardware to do different tasks.

A key part of the code is the 500ms delay in the Arduino loop. While this delay is necessary to keep the serial buffer clean and stop data from flooding the port, it creates a small lag in the system. If you turn the potentiometer too fast, the LED response feels a bit delayed because of this timing gap.

Limitations and Sources of Error

- **COM Port Conflicts:** A common issue we ran into was the serial port getting blocked. Since Windows only allows one program to use a COM port at a time, leaving the Arduino IDE Serial Monitor open caused the Python script to crash with an error. We have to make sure the Serial Monitor is closed before running Python.

Conclusion

The experiment successfully met its objectives by setting up stable, bidirectional serial communication between the Arduino Uno and a PC running Python via USB. We verified that data could flow both ways: the Arduino sent raw potentiometer readings to Python, and Python sent control signals back to toggle the LED based on the half-scale threshold logic ($ADC > 512$).

Recommendations

To improve the system behavior and prevent common errors in future mechatronics lab sessions, the following steps are recommended:

- **Incorporate Robust Error Catching in Python:** If a packet gets corrupted mid-stream or a non-numeric character is captured, it can break the parsing logic. Adding try-except blocks specifically around individual list indices would make the script more robust against serial noise.
- **Improve Safety Handling with Software Lockouts:** Currently, if an unexpected string is accidentally passed down the serial buffer, the microcontroller does not have a default fail-safe state handler. Adding an explicit conditional check that resets all output values to low when unrecognized data is parsed would prevent the hardware from locking into undefined states.

References

- <https://lastminuteengineers.com/servo-motor-arduino-tutorial/>

Acknowledgments

All praise belongs to Allah SWT for granting us the strength and wisdom to complete the "Serial Communication between Arduino and Python" experiment. We are grateful for His guidance in navigating the challenges faced throughout the circuit integration process. We also wish to acknowledge the vital support of our lecturers and fellow peers. Their academic supervision and practical insights into mechatronic system integration were essential to the successful operation of our project and the completion of this report.

Student's Declaration

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or persons.

We hereby certify that this report has not been done by only one individual and all of us have contributed to the report.

We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We therefore, agreed unanimously that this report shall be submitted for marking and this final printed report has been verified by us.

Signature: 

Read [/]

Name: Muhammad Aqeel Raziq Bin Mohamad Azhan

Understand [/]

Matric Number: 2314077

Agree [/]

Signature: 

Read [/]

Name: Muhammmad Aiman Bin Mohd Khairunnizam

Understand [/]

Matric Number: 2226273

Agree [/]

Signature: 

Read [/]

Name: Muhammad Aqil Syamil bin Abdul Rahim

Understand [/]

Matric Number: 2416245

Agree [/]